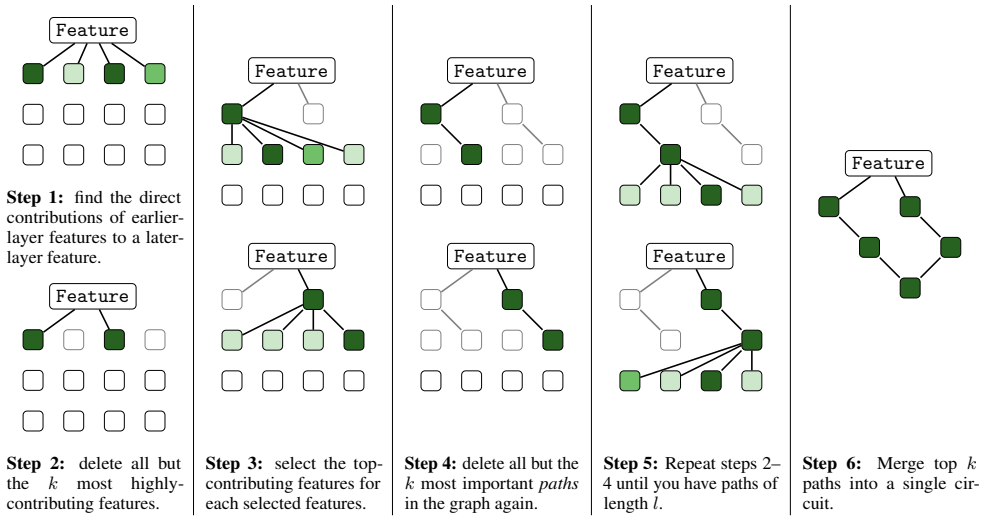

Figure 2: A visualization of the circuit-finding algorithm.

determine which edges are included in this subgraph, we thus have to compute **attributions** for each edge: how much the earlier node contributes to the later node's own contribution. Circuit analysis with SAEs thus entails computing the attribution of pre-MLP SAE features to post-MLP SAE features, *as mediated through the MLP.* Standard methods for computing attributions are causal patching (which inherently only gives information about local MLP behavior on a single input) and methods like input-times-gradient or attribution patching (which are equivalent in this setting). We will now demonstrate why these methods cannot yield information about the MLP's general behavior. Letting $\mathbf{z}$ be the activation of an earlier-layer feature, $\mathbf{z}'$ be the activation of the later-layer feature, and $\mathbf{y}$ be the activation of the MLP at layer l', the input-times-gradient is given by:

$$\mathbf{z}\left(\frac{\partial \mathbf{z}'}{\partial \mathbf{z}}\right) = \mathbf{z}\left(\frac{\partial \mathbf{z}'}{\partial \mathbf{y}}\frac{\partial \mathbf{y}}{\partial \mathbf{z}}\right). \tag{6}$$

Unfortunately, not only is $\mathbf{z}$ input-dependent, but so is $\frac{\partial \mathbf{z}'}{\partial \mathbf{z}}$ as well, because $\frac{\partial \mathbf{y}}{\partial \mathbf{z}}$ is.

This means that we cannot use SAEs to understand the general behavior of MLPs on various inputs. In contrast, we will show that when we replace MLP sublayers with sufficiently faithful and interpretable transcoders, we obtain attributions that neatly factorize into *input-dependent* terms and *input-invariant* terms; the latter can be computed just from model and transcoder weights, and tell us about the MLP behavior across all inputs.

3.2.1 Attribution between transcoder feature pairs

We begin by showing how to compute attributions between pairs of transcoder features. This attribution is given by the product of two terms: the earlier feature's activation (which depends on the input to the model), and the dot product of the earlier feature's decoder vector with the later feature's encoder vector (which is independent of the model input).

The following is a more formal restatement. Let $z_{TC}^{(l,i)}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right)$ denote the scalar activation of the i-th feature in the layer l transcoder on token t, as a function of the MLP input $\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}$ at token t in layer l. Then for layer $l < l'$, the contribution of feature i in transcoder l to the activation of feature i' in transcoder l' on token t is given by

$$\underbrace{z_{TC}^{(l,i)}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right)}_{\text{input-dependent}}\underbrace{\left(\mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{i})}\cdot\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')}\right)}_{\text{input-invariant}} \tag{7}$$

This expression is derived in App. D.2. Note that $\left(\mathbf{f}_{\mathbf{dec}}^{(\mathbf{l},\mathbf{i})}\cdot\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')}\right)$ is *input-invariant*: once the transcoders have been trained, this term does not depend on the input to the model. This term,

analyzed in isolation, can thus be viewed as providing information about the general behavior of the model. The only *input-dependent* term is $z_{TC}^{(l,i)}\left(\mathbf{x}_{\mathbf{mid}}^{(\mathbf{l},\mathbf{t})}\right)$, the activation of feature i in the layer l transcoder on token t. As such, this expression cleanly factorizes into a term reflecting the general input-invariant connection between the pair of features and an interpretable term reflecting the importance of the earlier feature on the current input.

3.2.2 Attribution through attention heads

So far, we have addressed how to find the attribution of a lower-layer transcoder feature directly on a higher-layer transcoder feature at the same token. But transcoder features can also be mediated by attention heads. We will thus extend the above analysis to account for finding the attribution of transcoder features through the OV circuit of an attention head. For a full derivation, see App. D.3.

As before, we want to understand what causes feature i' in the layer l' transcoder to activate on token t. Given attention head h at layer l with $l < l'$, the contribution of token s at layer l through head h to feature i' in layer l' at token t is given by

$$\mathrm{score}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}, \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right)\left(\left(\left(\mathbf{W}_{\mathbf{OV}}^{(\mathbf{l},\mathbf{h})}\right)^{T}\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l}',\mathbf{i}')}\right)\cdot\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right), \tag{8}$$

where $\mathrm{score}^{(l,h)}\left(\mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{t})}, \mathbf{x}_{\mathbf{pre}}^{(\mathbf{l},\mathbf{s})}\right)$ is the attention score for head h and layer l from token s to token t.

3.2.3 Finding computational subgraphs

Using this observation, we present a method for finding computational subgraphs. We now know how to determine, on a given input and transcoder feature i', which earlier-layer transcoder features i are important for causing i' to activate. Once we have identified some earlier-layer features i that are relevant to i', then we can then recurse on i to understand the most important features causing i to activate by repeating this process.

Doing so iteratively (and greedily pruning all but the most important features at each step) thus yields a set of computational paths (a sequence of connected edges). These computational paths can then be combined into a computational subgraph, in such a way that each node (transcoder feature or attention head), edge, and path is assigned an attribution. A full description of the circuit-finding algorithm is presented in App. D.5. Figure 2 provides a visualization of this algorithm.

3.2.4 De-embeddings: a special case of input-invariant information

Earlier, we discussed how to compute the input-invariant connection between a pair of transcoder features, providing insights on general behavior of the model. A related technique is something that we call **de-embeddings**. A de-embedding vector for a transcoder feature is a vector that contains the direct effect of *the embedding of each token in the model's vocabulary* on the transcoder feature. The de-embedding vector for feature i in the layer l transcoder is given by ${\mathbf{W}_{\mathbf{E}}}^{T}\mathbf{f}_{\mathbf{enc}}^{(\mathbf{l},\mathbf{i})}$, where $\mathbf{W}_{\mathbf{E}}$ is the model's token embedding matrix. Importantly, this vector gives us input-invariant information about how much each possible input token would directly contribute to the feature's activation.

Given a de-embedding vector, looking at which tokens in the model's vocabulary have the highest de-embedding scores tells us about the feature's general behavior. For example, for a certain GPT2-small MLP0 transcoder feature that we investigated, the tokens with the highest scores were `oglu`, `owsky`, `zyk`, `chenko`, and `kowski`. Notice the interpretable pattern: all of these tokens come from European surnames, primarily Polish ones, suggesting that the feature generally fires on Polish surnames.

4 Comparison with SAEs

Transcoders were originally conceived as a variant of SAEs, and as such, there are many similarities between them. They differ only in their training objective: because SAEs are autoencoders, the faithfulness term in the SAE loss measures the reconstruction error between the SAE's output and its original input. In contrast, the faithfulness term of the transcoder loss measures the error between the transcoder's output and the original MLP sublayer's output.

Table 1: The number of interpretable features, possibly-interpretable features, and uninterpretable features for the transcoder and MLP-in SAE. Of the interpretable features, we additionally deemed 6 transcoder features, and 16 SAE features to be "context-free", meaning they appeared to fire on a single token without any evident context-dependent patterns.

	Transcoder	MLP-in SAE
# interpretable	41	38
# maybe	8	8
# uninterpretable	1	4

Because of these similarities, SAEs can be quantitatively evaluated (for sparsity and fidelity) and qualitatively evaluated (for feature interpretability) in precisely the same way as transcoders, using standard SAE evaluation methods [4, 29]. We now report the results of evaluations comparing SAEs to transcoders on these metrics, and find that transcoders are comparable to or better than SAEs.

4.1 Blind interpretability comparison of transcoders to SAEs

In order to evaluate the interpretability of transcoders, we manually attempted to interpret 50 random features from a Pythia-410M layer 15 transcoder and 50 random features from a Pythia-410M layer 15 SAE trained on *MLP inputs*.[3] For each feature, the examples in a subset of the OpenWebText corpus that caused the feature to activate the most were computed ahead of time. Then, the features from both the SAE and the transcoder were randomly shuffled. For each feature, the maximum-activating examples were displayed, but not whether the feature came from an SAE or transcoder. We recorded for each feature whether or not there seemed to be an interpretable pattern, and only after examining every feature did we look at which features came from where. The results, shown in Table 1, suggest transcoder features are approximately as interpretable as SAE features. This further suggests that transcoders incur no penalties compared to SAEs.

4.2 Quantitative comparison of transcoders to SAEs

4.2.1 Evaluation metrics

We evaulate transcoders *qualitatively* on their features' interpretability as judged by a human rater, and *quantitatively* on the sparsity of their activations and their fidelity to the original MLP's computation.

As a qualitative proxy measure for the interpretability of a feature, we follow Bricken et al. [7] in assuming that interpretable features should demonstrate interpretable patterns in the examples that cause them to activate. To this end, one can run the transcoder on a large dataset of text, see which dataset examples cause the feature to activate, and see if there is an interpretable pattern among these tokens. While imperfect [6], this is still a reasonable proxy for an inherently qualitative concept.

To measure the sparsity of a transcoder, one can run the transcoder on a dataset of inputs, and calculate the mean number of features active on each token (the mean L_0 norm of the activations). To measure the fidelity of the transcoder, one can perform the following procedure. First, run the original model on a large dataset of inputs, and measure the next-token-prediction cross entropy loss on the dataset. Then, replace the model's MLP sublayer corresponding to the transcoder *with the transcoder*, and measure the modified model's mean loss on the dataset. Now, the faithfulness of the transcoder can be quantified as the difference between the modified model's loss and the original model's loss.

4.2.2 Results

We trained SAEs and transcoders on activations from GPT2-small [44], Pythia-410M, and Pythia-1.4B [2]. For each model, we trained multiple SAEs and transcoders on the same inputs, but with different values of the λ_1 hyperparameter controlling the fidelity-sparsity tradeoff for each SAE and each transcoder. The transcoders were trained on MLP-in and MLP-out activations, while SAEs were

[3] We used SAEs trained on MLP inputs here because the interpretability case studies look at feature activations, which are solely dependent on the encoder vectors of the SAEs and transcoders. Because transcoders' encoder vectors live in MLP input space, we thought that the comparison would be most accurate if our SAEs' encoder vectors also lived in MLP input space.